2A: Trees for service regime changes

Problem statement and worked resolution

Companion to `02_A_service_regimes.ipynb`

How to use this explanation. The resolution connects a manual one-split search to fitted trees, pruning, and stability. Each major function is explained where it enters the calculation. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Problem statement and definitions

For 1,200 independent requests, generate payload $s \sim U(0, 10)$ MB, queue length $q \sim \text{Poisson}(3)$, and independent noise $\epsilon \sim N(0, 8^2)$ ms. Set

$$y = 40 + 20 \cdot \mathbf{1}\{q > 3\} + 30 \cdot \mathbf{1}\{q > 3 \text{ and } s > 5\} + \epsilon.$$

The indicator $\mathbf{1}\{\cdot\}$ is 1 if the condition holds and 0 otherwise. The dot means multiplication. The conditional mean is therefore 40 ms for $q \leq 3$, 60 ms for $q > 3, s \leq 5$, and 90 ms for $q > 3, s > 5$. The question is whether trees capture these jumps better than a linear plane, and how tree size and training-sample changes affect the answer.

A tree's internal nodes ask threshold questions; its terminal nodes are leaves. A regression leaf predicts the mean target among its fitting rows. A stump has one split and two leaves. Residual sum of squares (RSS) is total squared deviation from predictions; root mean squared error (RMSE) is an average-based prediction metric in milliseconds. Cross-validation (CV) evaluates fits on held-out training folds.

1 Verify a regression stump

For a node containing row indices A, the best constant prediction under squared loss is

$$\bar{y}_A = \frac{1}{|A|} \sum_{i \in A} y_i, \qquad \text{RSS}(A) = \sum_{i \in A} (y_i - \bar{y}_A)^2.$$

For feature j and threshold t, define $A_L = \{i : x_{ij} \leq t\}$ and $A_R = \{i : x_{ij} > t\}$. The split minimizes $\text{RSS}(A_L) + \text{RSS}(A_R)$ over admissible children. Minimum leaf size rules out children with too few rows.

Main Python calls

The custom `stump_candidates` returns a table of all admissible feature/threshold pairs and their child RSS totals. `np.argsort` orders row positions by a feature. `np.cumsum` supplies prefix sums. `idxmin` selects the first smallest score. After sorting, test midpoints between distinct adjacent values. Equal values cannot be separated by a threshold, so skip them. For sorted responses, prefix sum S_k and squared sum Q_k give

$$\text{RSS}(A_L) = Q_k - S_k^2/k.$$

The right side uses total sums minus left sums. The code subtracts the overall mean before computing these quantities to reduce numerical cancellation; translation leaves RSS unchanged. Tiny negative values caused by roundoff are clipped to zero.

For $x = (1, 2, 3, 4, 5, 6)$ and $y = (2, 3, 2, 8, 9, 10)$:

Threshold	1.5	2.5	3.5	4.5	5.5
Child RSS	53.2	39.25	8/3	25.25	46.8

The threshold 3.5 gives left mean 7/3 and right mean 9. Left RSS is 2/3, right RSS is 2, so total RSS is 8/3. The notebook verifies every candidate against directly calculated residual sums, and checks repeated feature values separately. The manual search breaks exact ties by feature order then threshold order; agreement with the library here is checked on predictions rather than assumed for all possible tied data sets.

2 Generate requests and reserve partitions

`np.random.default_rng(17)` generates features and noise; `train_test_split` with seed 23 yields 720 training, 240 validation, and 240 test rows. The first split reserves 20%; the second reserves 25% of the remaining 80%, again 20% of the total.

In NumPy, multiplication treats Boolean conditions as zeros and ones, and `&` combines conditions element by element. Thus the supplied code implements the indicator formula directly. Only payload and queue form `X_train`; simulated mean latency is not a feature. The training scatterplots mix the other variable's values. At one payload size, multiple mean levels may appear because queue length differs. The two-dimensional structure therefore matters.

3 Fit the basic candidates and compare surfaces

Model call	Meaning
`DummyRegressor(strategy="mean")`	Learns the training mean and predicts it everywhere; a reference that ignores the features.
`LinearRegression()`	Fits $b + w_s s + w_q q$; it cannot reproduce sharp jumps exactly.
`DecisionTreeRegressor(max_depth=1)`	A stump with at most one decision along a path.
`DecisionTreeRegressor(max_depth=2)`	At most two decisions along a path, allowing up to four leaves.

All four objects use `fit(X_train,y_train)` and `predict(X_new)`. Trees use squared-error splitting; `random_state` makes randomized feature ordering and ties reproducible. Depth is not the total number of nodes.

The manual search chooses queue ≤ 3.5, equivalent to $q \leq 3$ for integer queues. The two leaf predictions are about 39.5825 and 73.9255 ms. The high-queue leaf averages the 60 and 90 ms groups because a stump has no second decision. `np.where` constructs manual predictions, checked against the fitted library stump.

`meshgrid` builds fixed payload/queue pairs, `column_stack` turns them into feature rows, and `pcolormesh` displays predicted latency as color. All panels share a color scale. Neither the grid nor the true mean is used for fitting or selection. A depth-two tree can express the three regimes, but its fourth leaf may fit noise in the low-queue region. A representational capacity is not a promise that every possible split is useful.

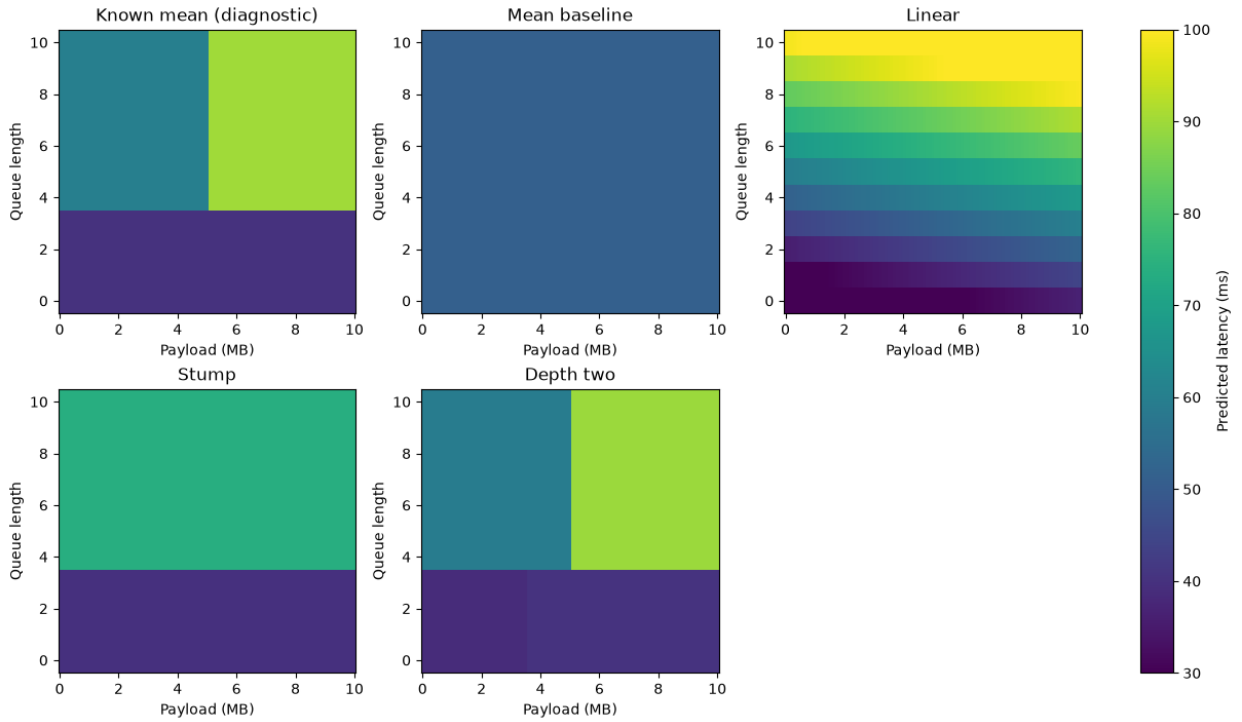


Figure 1: The baseline is flat, the linear fit is a plane, and tree predictions are constant within rectangular regions. The known mean is a simulation diagnostic.

4 Select a pruning penalty using training CV

Grow trees with `min_samples_leaf=2` and no depth limit, then prune using `ccp_alpha`. For a tree T fit on n_{fit} rows, the criterion is

$$R_\alpha(T) = \frac{1}{n_{\text{fit}}} \sum_{\ell \in \text{leaves}(T)} \text{RSS}(A_\ell) + \alpha |\text{leaves}(T)|.$$

The first term is training mean squared error; the second charges for leaves. Larger α favors a smaller tree. It has squared-response units under this normalization. Depth restriction prevents growth beyond a limit; cost-complexity pruning removes branches according to their error reduction and size cost.

`KFold` creates five fitting/scoring index pairs from the 720 training rows. Each fit uses 576 rows and each held-out score uses 144. Search $\alpha \in \{0, 0.01, 0.1, 1, 5, 10, 25, 50, 100\}$, average five held-out RMSE scores, and select the first smallest mean. A separate full-training tree supplies the training-error and leaf-count plots for each alpha.

Alpha	Training RMSE	CV RMSE	Full-training leaves
0	4.458	10.341	311
0.1	5.152	9.949	153
1	7.849	7.868	3
100	11.826	11.837	2

Alphas 1, 5, 10, 25, and 50 tie in the saved CV results; the selection rule chooses 1. Fit it again on all 720 rows, giving three leaves. Fold standard deviation describes variability, not an interval for population error.

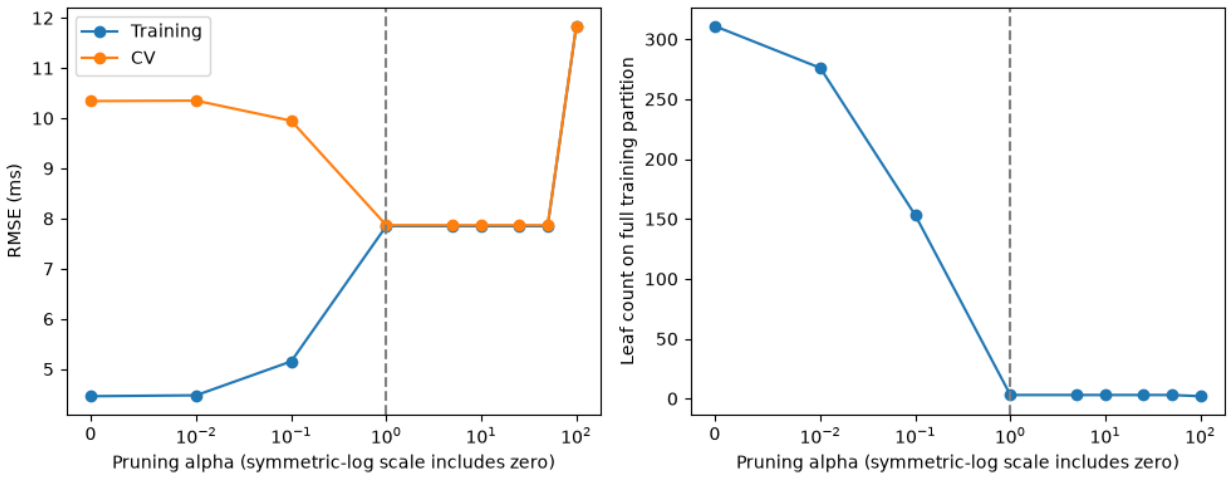


Figure 2: The unpruned model achieves low training error but worse held-out error. Moderate pruning recovers a small tree; excessive pruning removes a useful distinction.

5 Bootstrap stability at boundaries and fixed inputs

Draw 30 resamples of 720 training rows with replacement. Some original rows repeat and others disappear. Refit the depth-two model and the selected pruning configuration, keeping alpha fixed. These fits are diagnostics and do not replace the original final models.

Main Python calls

`rng.integers` draws bootstrap positions. The fitted structure is stored in `tree_`: its `feature` and `threshold` arrays identify splits, and `node_count` gives the number of nodes. Negative feature indices identify leaves. `crosstab` counts root-feature choices. Grouping with `groupby` and summarizing with `agg` produces the threshold and prediction tables. All 30 resamples use queue at the root for both configurations. Nevertheless, predictions at payload 5.01 MB and queue 4 have standard deviations about 12.91 ms (depth two) and 12.77 ms (pruned). At payload 4.99 MB and queue 4, they are only about 0.56 and 0.59 ms in this particular run. This asymmetry is possible: the distribution of fitted thresholds relative to the two selected points determines which point frequently changes leaf. Away from boundaries, routing is steadier. The pooled threshold table includes all splits by feature, including noise-fitting splits; its minimum and maximum do not track the same branch across resamples. Bootstrap summaries are not formal confidence intervals or independent test data.

6 Validation audit and final test evaluation

Validation checks all five prespecified fits without changing alpha. Final test metrics then compare the unchanged fits:

Model	RMSE (ms)	MAE (ms)	Depth	Leaves
Mean baseline	19.101	15.316	—	—
Linear	13.634	10.867	—	—
Stump	11.414	8.897	1	2
Depth two	8.249	6.474	2	4
Pruned	8.274	6.489	2	3

`mean_squared_error` followed by a square root gives RMSE; `mean_absolute_error` gives MAE. `get_depth` and `get_n_leaves` report structure, applicable only to trees. `perf_counter` measures prediction time for the same 240-row batch, repeated 30 times and summarized by the median. These times include call overhead and depend on the machine; repeating prediction does not refit a model or select a new one.

Answers to the interpretation questions

The linear plane cannot represent discontinuous regime boundaries. Lower training error does not imply better new-request prediction: the large unpruned tree fits noise. Pruning attains nearly the same test error as the depth-two model with fewer leaves; their tiny difference should not trigger test-based selection. Inputs close to learned boundaries can have unstable predictions despite stable root features. Finally, the exact conditional mean has population RMSE 8 ms because it cannot predict independent noise. This simulation favors axis-aligned trees; it is not a universal comparison against linear models.

Further explanation: how a regression tree makes progress

Derive the split gain instead of memorizing it

Let $n_A = n_L + n_R$. Within either child, write $y_i - \bar{y}_A = (y_i - \bar{y}_{\text{child}}) + (\bar{y}_{\text{child}} - \bar{y}_A)$. The cross-terms sum to zero because child residuals sum to zero. Consequently,

$$\text{RSS}(A) = \text{RSS}(A_L) + \text{RSS}(A_R) + n_L(\bar{y}_L - \bar{y}_A)^2 + n_R(\bar{y}_R - \bar{y}_A)^2.$$

Substituting the weighted parent mean simplifies the reduction to

$$\text{RSS}(A) - \text{RSS}(A_L) - \text{RSS}(A_R) = \frac{n_L n_R}{n_A}(\bar{y}_L - \bar{y}_R)^2.$$

This explains why child means must differ for a useful split, and why a tiny isolated group has less influence than a substantial group with the same mean difference. It also explains why taking an unweighted average of child mean squared errors generally chooses the wrong criterion.

Define node impurity $I(A) = \text{RSS}(A)/n_A$. Then the generic weighted impurity decrease is $\Delta I = I(A) - (n_L/n_A)I(A_L) - (n_R/n_A)I(A_R)$, exactly the RSS reduction divided by n_A. RSS and impurity are therefore related but not interchangeable quantities: one is a total and one an average.

Why an extra leaf can be attractive during training

Suppose splitting one leaf reduces normalized training squared error by 0.6 and adds one leaf. The change in the cost-complexity objective is $-0.6 + \alpha$. At $\alpha = 0.1$ that split is worth keeping under this local comparison; at $\alpha = 1$ it is not. More generally, compare error reduction with the increase in leaf count. Actual cost-complexity pruning compares whole subtrees along a pruning sequence, so this one-leaf example explains the tradeoff without claiming that every branch is independently pruned.

The low-queue regime has a constant true mean, yet splitting it can always exploit accidental target variation. Training data therefore reward some useless subdivisions. Validation on excluded rows is what distinguishes those accidental improvements from useful structure. Minimum leaf size and maximum depth constrain growth; pruning applies an error-versus-complexity tradeoff after a tree has been grown under those constraints.

A discontinuity explains bootstrap instability

For a fitted split near $s = 5$ at high queue, changing the threshold from 5.00 to 5.02 sends the fixed input $s = 5.01$ from the high-latency leaf to the lower-latency leaf. A threshold movement of only 0.02 MB can therefore change prediction by roughly 30 ms. This is not a numerical failure: it is how a piecewise constant rule behaves. A model can have small average test error while being unstable at a few boundary inputs.

Conversely, the mean of a large fixed group has noise variance approximately $8^2/n_{\text{leaf}}$ in this simulator. Tree leaves are selected from data, so this simple formula is not a ready-made confidence interval for a fitted tree, but it explains the general stabilizing role of larger groups.

Documentation details and practical checks

The DecisionTreeRegressor reference describes depth and leaf constraints, pruning, and the fitted tree attributes. In particular, minimum leaf size is not a target number of leaves. A depth-two tree has at most four leaves, not exactly two. Its input processing uses floating-point representations, so near-ties and extremely close thresholds need not match a hand calculation bit for bit. Compare predictions and objective values within tolerance.

Tips for reading the figures. A boundary at queue 3.5 is a separator between integer values 3 and 4, not a claim that half a request can be queued. All contour panels must use the same color scale for fair visual comparison. Pooled bootstrap threshold ranges include different branches: a very small minimum payload threshold does not mean the main boundary wandered from 5 to that value. Finally, prediction-time differences for tiny batches may reflect overhead; report batch size and hardware context before making an efficiency claim.